

MODULE 6

Claude Dynamic Workflows — Complete Guide

JavaScript-orchestrated parallel subagent execution, plan-as-code, Ultracode,
/deep-research, cost management (released May 28 2026)

NEW — Released May 28, 2026 | Max · Team · Enterprise plans

Claude Certified Architect (CCA-F) | Professional Enterprise Architect | May 2026

What You Will Master in This Module

- What Dynamic Workflows are — how they differ from subagents, skills, and MCP
- Architecture: Claude writes workflow.js, Bun runtime executes it, agents work in parallel
- Plan-as-code: why plans live in code (not context) — the scaling breakthrough
- Triggering: 'workflow' keyword, Ultracode mode, /deep-research built-in command
- Technical limits: 16 concurrent agents, 1,000 total per run, plan constraints
- Pause, resume, and checkpoint behavior for interrupted long-running workflows
- workflow.js structure: fan-out, verification, convergence, synthesis phases
- Cost modeling: estimating token spend for 200-1,000 agent workflow runs
- Availability: Claude Code v2.1.154+, Max/Team/Enterprise plans, all cloud platforms
- Decision guide: Workflows vs Subagents vs Skills vs MCP — when to use each

6.1 What Are Dynamic Workflows?

Released May 28, 2026 alongside Claude Opus 4.8, Dynamic Workflows are a fundamental architectural shift in how Claude Code handles large-scale parallel tasks. They solve the context window scaling ceiling that limited previous multi-agent approaches.

Feature	Old: Subagents in context	New: Dynamic Workflows
Plan location	Claude's 200K context window	JavaScript code (workflow.js) — outside context
Intermediate results	Accumulate in Claude's context	Live in JS variables — never touch context
Final result in session	Everything accumulates	Only the synthesized final answer
Max parallel agents	Context-limited (~5-10 practical)	16 concurrent, 1,000 total per run
Resumability	No — restart on interruption	Yes — checkpointed after each agent
Context pollution	Every agent result uses context	Zero — context stays clean throughout
Available on	All Claude Code plans	Max, Team, Enterprise plans only
Trigger method	Explicit Task tool call	'workflow' keyword or Ultracode mode

6.2 Architecture & Execution Flow

```
USER PROMPT: 'Create a workflow to audit 200 microservices for security issues' STEP 1: CLAUDE PLANS
■■ Claude writes workflow.js — a JavaScript orchestration script Plan includes: task decomposition,
agent prompts, schemas, concurrency settings STEP 2: BUN RUNTIME EXECUTES ■■ Bun runtime takes over
— your session stays responsive Runtime manages: agent spawning, concurrency (max 16),
checkpointing, retries Note: workflow.js CANNOT directly access filesystem/shell — only agents can
STEP 3: PARALLEL AGENT EXECUTION ■■ [Agent 1]: audit service-auth → result stored in JS variable
■■ [Agent 2]: audit service-payments → result stored in JS variable ■■ [Agent 3]: audit
service-users → result stored in JS variable ■■ ... (up to 16 concurrent) ■■ Progress checkpointed
after each completion STEP 4: VERIFICATION (adversarial agents challenge findings) ■■ Agents
assigned to refute high-severity findings Findings that survive challenge = high confidence STEP 5:
SYNTHESIS ■■ Single synthesis agent creates final report Only this final report is returned to your
session Session context remains clean throughout entire run
```

6.3 Technical Specifications

Orchestration language	JavaScript — Bun runtime. Claude writes workflow.js using standard JS constructs.
Concurrent agents (max)	16 agents running simultaneously. Queue-based: next starts when one completes.
Total agents per run	1,000 agents hard limit. Plan your token budget accordingly.
Context per agent	Each agent has its own fresh 200K context window. Intermediate results in JS vars.
Script restrictions	workflow.js cannot access filesystem or shell directly — only agents can use tools.
Resumability	Checkpointed after each agent. Interrupted workflows resume within same session.

Retry behavior	Failed agents auto-retried with exponential back-off. Configurable retry count.
Trigger methods	(1) Include 'workflow' in prompt. (2) Enable Ultracode setting. (3) Use /deep-research.
Ultracode mode	Combines xhigh reasoning effort + automatic workflow orchestration for complex tasks.
Built-in workflows	/deep-research: fan-out research → cross-validation → synthesis report.
Required version	Claude Code v2.1.154+. Check with: claude --version
Available plans	Max 5x, Max 20x, Team Standard, Team Premium, Enterprise (admin must enable).
Cloud platforms	CLI, Desktop, VS Code. Also: Claude API, Amazon Bedrock, Vertex AI, MS Foundry.

6.4 workflow.js — Structure & Example

```
// workflow.js – Auto-generated by Claude for security audit // Executed by Bun runtime –
filesystem/shell only accessible via agents const { runAgent, parallel, sequential } = await
import('@anthropic/claude-code-runtime'); // Phase 1: Discovery – find all service directories const
discovery = await runAgent({ prompt: 'List all microservice directories in this monorepo. Return
JSON: {services:[string[]]}', tools: ['Read','Glob'], outputSchema: { services: ['string'] }, model:
'claude-haiku-4-5-20251001' // Cheap for discovery }); // Phase 2: Fan-out – audit each service in
parallel const audits = await parallel( discovery.services.map(service => ({ prompt: `Security audit
of ${service}: check SQL injection, auth bypass, hardcoded secrets, input validation. Return JSON:
{service:string, risk:'high'|'medium'|'low',
findings:[{type,file,line,description,recommendation}]}`, tools: ['Read','Grep'], budget: {
maxTokens: 4000 }, // Per-agent token budget retries: 2, model: 'claude-sonnet-4-6' })), {
concurrency: 16 } // Run up to 16 simultaneously ); // Phase 3: Verification – challenge high-risk
findings const highRisk = audits.filter(a => a.risk === 'high'); const verified = await parallel(
highRisk.map(audit => ({ prompt: `Verify this finding. Check for false positives:
${JSON.stringify(audit)} Return: {confirmed:bool, confidence:0-100, reasoning:string}`, tools:
['Read'], model: 'claude-sonnet-4-6' }))) ); // Phase 4: Synthesis – one agent creates the executive
report const report = await runAgent({ prompt: `Synthesize verified security findings into an
executive report. All findings: ${JSON.stringify(verified)} Include: risk summary, top 10 critical
findings, remediation roadmap.` , tools: [], // Synthesis agent needs no tools model:
'claude-opus-4-8' // Best model for synthesis }); return report; // Only this returns to the user
session
```

6.5 Cost Management & Tool Comparison

Warning: Workflows can spawn up to 1,000 agents. At Sonnet 4.6 pricing (\$3/\$15/MTok), a large workflow can cost \$100-500+. Always estimate cost and set per-agent token budgets before running at scale.

Cost Estimation Template

```
# 200-service security audit cost estimate pricing = {'haiku':(1,5), 'sonnet':(3,15), 'opus':(5,25)}
# $/MTok in/out phases = [ ('discovery', 1, 'haiku', 5_000, 500), ('audit', 200, 'sonnet', 20_000,
2_000), ('verify', 40, 'sonnet', 8_000, 1_000), ('synthesis', 1, 'opus', 60_000, 10_000), ] total =
sum(n * (ti*p[0] + to*p[1]) / 1e6 for _, n, m, ti, to in phases for p in [pricing[m]])
print(f'Estimated total: ${total:.2f}') # ~$136
```

When to Use Each Agentic Tool

Tool	Use When	Example
Dynamic Workflows	Many parallel independent tasks (50+); intermediate res	Sts would fill context and have ability to interact across
Subagents (Task)	Single focused isolated task; security boundary needed	Delegate security easily of one PR to a read-only agent
Skills (SKILL.md)	Reusable task playbook; consistent execution each time	And often forked; all the same 6-step process every tim
MCP Servers	Connecting to external services (CRM, GitHub, Gmail);	Only satisfied across many sessions, read Gmail

New Capability: Dynamic Workflows transform Claude Code from a context-constrained assistant into a massively parallel compute platform. Anthropic used hundreds of parallel agents in the Claude 4.8 internal migration — this architecture is production-validated at scale.